Problem Sheet 04

1. This exercise demonstrates how to implement a simple database in Pro log. Copy the following list of eight facts (the data) into a program.

```
born(jan, date(20,3,1977)).
born(jeroen, date(2,2,1992)).
born(joris, date(17,3,1995)).
born(jelle, date(1,1,2004)).
born(jesus, date(24,12,0)).
born(joop, date(30,4,1989)).
born(jannecke, date(17,3,1993)).
born(jaap, date(16,11,1995)).
```

That is, we are representing dates as terms of the form `date(Day,Month,Year).`

1) Write a predicate `year/2` to retrieve all people born in a given year (through repeated backtracking). Example:

```
?- year(1995, Person).
Person = joris ;
Person = jaap ;
No
```

2) Implement a predicate `before/2` that, when given two date-expressions, will succeed if the first expression represents a date before the date represented by the second expression (you may assume that the user will only ask for well-formed dates, e.g., not for the 31st of April, and so forth). Example:

```
?- before(date(31,1,1990), date(7,7,1990)).
Yes
```

3) Implement a predicate `older/2` that succeeds in case the person given first is (strictly) older than the person given second. Example:

```
?- older(jannecke, X).
X = joris ;
X = jelle ;
X = jaap ;
No
```

You should get 28 solutions for the query older(X, Y). Explain why.

2. Polynomials can be represented as lists of pairs of coefficients and exponents. For example, the polynomial

   $4x^5 + 2x^3 - x + 27$

   can be represented as the following Prolog list:

   ```
   [(4,5), (2,3), (-1,1), (27,0)]
   ```

   Write a Prolog predicate `poly_sum/3` for adding two polynomials using that representation. Try to find a solution that is independent of the ordering of pairs inside the two given lists. Likewise, your output doesn't have to be ordered. Examples:

   ```
   ?- poly_sum([(5,3), (1,2)], [(1,3)], Sum).
   Sum = [(6,3), (1,2)]
   Yes
   ?- poly_sum([(2,2), (3,1), (5,0)], [(5,3), (1,1),
   (10,0)], X).
   X = [(4,1), (15,0), (2,2), (5,3)]
   Yes
   ```

   Hints: Before you even start thinking about how to do this in Prolog, recall how the sum of two polynomials is actually computed. A rather simple solution is possible using the built-in predicate `select/3`. Note that the list representation of the sum of two polynomials that don't share any exponents is simply the concatenation of the two lists representing the arguments.

3. Recall that the set of prime numbers is {2,3,5,7,11,13,17,… } i.e., the set of numbers with exactly two divisors each (namely 1 and the number itself). Write a Prolog predicate `prime/1` to check whether given number is prime. Examples:

   ```
   ?- prime(17).
   Yes
   ?- prime(18).
   No
   ```

4. In 1742, in a letter to the famous mathematician Leonhard Euler, Christian Goldbach conjectured that every even integer greater than 2 can be expressed as the sum of two prime numbers. In his own words (quote taken from Wikipedia, consulted on 3 August 2015):

   > "Dass ... ein jeder numerus par eine summa duorum primorum sey, halte ich fur ein ganz gewisses theorema, ungeachtet ich dasselbe nicht demonstriren kann."

   To this date, nobody has been able to prove the truth of this statement for all integers, although it has been veri ed for very many of them with the help of computers. Write a predicate called `goldbach/2` that, when given an even integer greater than 2 in the first argument position, will return an expression of the form A + B, such that both A and B are prime numbers and their sum is equal to the input number. Examples:

   ```
   ?- goldbach(30, Solution).
   Solution = 7+23
   Yes
   ?- goldbach(17420000, Solution).
   Solution = 109+17419891
   Yes
   ```

   Start by implementing a predicate `prime/1` to test whether a given number is prime. Then think about what you need to do to find two prime numbers that add up to a given number N. First you need to choose the first number A, which can be any number between 2 and N/2 (think about why these are the correct bounds!). Then you need to check whether A really is prime. Then you need to compute B as the difference of N and A, and finally you also need to check whether B is prime. You may find the built-in predicate `between/3` useful.